Machine learning models generally do not require I/O operations; machine learning is limited by computation speed, so it benefits from true simultaneous computation with multiprocessing.

Large machine learning models all but require graphics processing units (GPUs) to perform efficiently, but distributed models on multiple CPUs can be competitive in some cases. Python provides a library called "multiprocessing" that makes multiprocessing very easy. Additionally, PyTorch wraps this library and has a method for allowing a model's parameters to be shared across multiple processes. Let's look at a simple example of multiprocessing.

As a contrived simple example, suppose we have an array with the numbers 0, 1, 2, 3 ... 64 and we want to square each number. Since squaring a number does not depend on any other numbers in the array, we can easily parallelize this across multiple processors.

Listing 5.1. Introduction to multiprocessing

```
import multiprocessing as mp
import numpy as np
def square(x):                                                    1
return np.square(x)
x = np.arange(64)                                                 2
>>> print(x)
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14,
15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48,
49, 50,
       51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63])
>>> mp.cpu_count()
    8
pool = mp.Pool(8)                                                 3
squared = pool.map(square, [x[8*i:8*i+8] for i in range(8)])      4
>>> squared
[array([ 0,  1,  4,  9, 16, 25, 36, 49]),
 array([ 64,  81, 100, 121, 144, 169, 196, 225]),
 array([256, 289, 324, 361, 400, 441, 484, 529]),
 array([576, 625, 676, 729, 784, 841, 900, 961]),
 array([1024, 1089, 1156, 1225, 1296, 1369, 1444, 1521]),
 array([1600, 1681, 1764, 1849, 1936, 2025, 2116, 2209]),
```